

AFRILANG Stdlib

std/c/statnormcdf

Bibliothèque AFRILANG `std/c/statnormcdf` (niveau complex, catégorie complex). Elle expose 6 fonction(s) : normPdf, norm

```
`import "std/c/statnormcdf" · `use statnormcdf`
```

- `normPdf(x number, mu number, sigma number) ? number`
- `normCdfApprox(x number, mu number, sigma number) ? number`
- `zFromP(p number) ? number`
- `standardize(x number, mu number, sigma number) ? number`
- `logLikelihood(data list number, mu number, sigma number) ? number`
- `bivariatePdf(x number, y number, rho number) ? number`

Category: complex | Tier: complex | Functions: 6

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG ``std/c/statnormcdf`` (niveau complex, catégorie complex). Elle expose 6 fonction(s) : `normPdf`, `norm`

```
`import "std/c/statnormcdf"` . `use statnormcdf`  
- `normPdf(x number, mu number, sigma number) ? number`  
- `normCdfApprox(x number, mu number, sigma number) ? number`  
- `zFromP(p number) ? number`  
- `standardize(x number, mu number, sigma number) ? number`  
- `logLikelihood(data list number, mu number, sigma number) ? number`  
- `bivariatePdf(x number, y number, rho number) ? number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/statnormcdf"  
use statnormcdf
```

Niveau : complex · Catégorie : Modules complex (c/)

Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? normPdf(x number, mu number, sigma number) ? number  
? normCdfApprox(x number, mu number, sigma number) ? number  
? zFromP(p number) ? number  
? standardize(x number, mu number, sigma number) ? number  
? logLikelihood(data list number, mu number, sigma number) ? number  
? bivariatePdf(x number, y number, rho number) ? number
```

4. Exemples d'utilisation

```
import "std/c/statnormcdf"  
use statnormcdf  
  
create result = normPdf(/* args */)   
say result  
  
create result = normCdfApprox(/* args */)   
say result  
  
create result = zFromP(/* args */)   
say result  
  
create result = standardize(/* args */)   
say result  
  
create result = logLikelihood(/* args */)   
say result  
  
create result = bivariatePdf(/* args */)   
say result
```

5. Bonnes pratiques

```
? Préférez `import "std/c/statnormcdf"` en tête de fichier.  
? Lancez `afrilang check` avant de livrer.  
? Combinez avec les modules stdlib de la même catégorie.  
? Voir /stdlib/ pour le source live et les démos playground.  
? Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe ? source

```
module statnormcdf  
  
function normPdf(x number, mu number, sigma number) returns number  
end
```

```
function normCdfApprox(x number, mu number, sigma number) returns number
end

function zFromP(p number) returns number
end

function standardize(x number, mu number, sigma number) returns number
end

function logLikelihood(data list number, mu number, sigma number) returns number
end

function bivariatePdf(x number, y number, rho number) returns number
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/statnormcdf` (tier complex, category complex). Exports 6 function(s): normPdf, normCdfApprox, zF

```
`import "std/c/statnormcdf"` . `use statnormcdf`  
- `normPdf(x number, mu number, sigma number) ? number`  
- `normCdfApprox(x number, mu number, sigma number) ? number`  
- `zFromP(p number) ? number`  
- `standardize(x number, mu number, sigma number) ? number`  
- `logLikelihood(data list number, mu number, sigma number) ? number`  
- `bivariatePdf(x number, y number, rho number) ? number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/statnormcdf"  
use statnormcdf
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? normPdf(x number, mu number, sigma number) ? number  
? normCdfApprox(x number, mu number, sigma number) ? number  
? zFromP(p number) ? number  
? standardize(x number, mu number, sigma number) ? number  
? logLikelihood(data list number, mu number, sigma number) ? number  
? bivariatePdf(x number, y number, rho number) ? number
```

4. Usage examples

```
import "std/c/statnormcdf"  
use statnormcdf  
  
create result = normPdf(/* args */)   
say result  
  
create result = normCdfApprox(/* args */)   
say result  
  
create result = zFromP(/* args */)   
say result  
  
create result = standardize(/* args */)   
say result  
  
create result = logLikelihood(/* args */)   
say result  
  
create result = bivariatePdf(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/statnormcdf"` at the top of your file.  
? Run `afirang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module statnormcdf  
  
function normPdf(x number, mu number, sigma number) returns number  
end
```

```
function normCdfApprox(x number, mu number, sigma number) returns number
end

function zFromP(p number) returns number
end

function standardize(x number, mu number, sigma number) returns number
end

function logLikelihood(data list number, mu number, sigma number) returns number
end

function bivariatePdf(x number, y number, rho number) returns number
end
end
```